

31/03/2026

Projet de structuration de données

Conception d'une BDD de gestion d'un magasin de végétaux

Mohamed Lamine Bah

UNIVERSTE CATHOLIQUE DE L'OUEST ANGERS

Table des matières

Projet de gestion d'une base de données :	1
I. Objectifs du projet :	1
II. Contraintes du projet :	1
III. Règles de métiers :	2
IV. Dictionnaire de données :	3
V. Modèle conceptuel de données :	3
VI. Modèle logique de données :	4
VII. Modèle physique de données :	4
VIII. Vérification et validation de la base de données :	9
IX. Conclusion :	13
X. Bibliographie :	13

Projet de gestion d'une base de données :

Dans le cadre d'un projet de structuration des données, il est proposé de concevoir un système d'information dédié à la gestion de stock d'un magasin de végétaux.

I. Objectifs du projet :

L'objectif principal de ce projet est de concevoir une base de données permettant d'assurer la gestion efficace du stock d'un magasin de végétaux. Cette base de données doit structurer l'ensemble des informations relatives aux produits (plantes, graines, engrais, accessoires), aux flux d'approvisionnement et de vente, ainsi qu'aux contraintes spécifiques au domaine végétal.

Les objectifs spécifiques sont les suivants :

- Centraliser les données relatives aux produits, catégories, espèces, fournisseurs, clients, achats, ventes, lots, stocks et emplacements de stockage.
- Garantir la traçabilité des mouvements de stock (entrées, sorties, pertes, inventaires) et des opérations associées.
- Respecter les spécificités métier des végétaux : péremption des lots, conditions de culture par espèce, localisation physique par emplacement.
- Produire un modèle normalisé, cohérent et évolutif, prêt pour la traduction en modèle logique de données (MLD) et implémentation SQL.

II. Contraintes du projet :

Le projet doit satisfaire les contraintes suivantes :

- Chaque produit doit obligatoirement appartenir à une seule catégorie et, s'il s'agit d'un végétal, à une seule espèce.
- Les relations n, doivent être résolues par des entités associatives.
- Non-négativité des stocks : aucune vente ne peut entraîner un stock négatif.

- Péremption des lots : un lot périmé ne peut plus être vendu et doit être sorti via un mouvement de type perte.
- Traçabilité complète : chaque LOT doit être rattachable à son fournisseur, son achat et ses mouvements associés.
- Localisation physique : chaque stock doit être associé à un emplacement précis.
- Historisation : toute opération (achat, vente) génère un mouvement de stock daté et tracé.
- Authentification : chaque opération est rattachée à un employé responsable.
- Le modèle doit permettre des requêtes rapides sur les stocks disponibles, alertes de réapprovisionnement et analyses de ventes.
- Évolutivité : possibilité d'ajouter de nouvelles catégories, espèces ou emplacements sans restructuration majeure.
- Normalisation : élimination des redondances et anomalies d'insertion/suppression.

L'ensemble du travail vise à proposer un modèle de données cohérent, normalisé et exploitable, capable de répondre aux besoins de gestion quotidienne du magasin et de garantir une meilleure traçabilité des opérations.

III. Règles de métiers :

Pour garantir l'intégrité et l'organisation, et éviter les erreurs dans notre base de données, nous allons intégrer des logiques opérationnelles qui sont entre autres :

1. Règles de classification des produits :

- Chaque produit doit être rattaché à une seule catégorie
- Un produit doit être associé à une seule espèce
- Les produits non végétaux ne sont pas rattachés à une espèce, ni à une catégorie

2. Règles d'approvisionnement :

- Un achat peut contenir plusieurs produits
- Un achat est toujours émis auprès d'un seul fournisseur, mais un fournisseur peut fournir plusieurs achats
- Une commande achat peut contenir plusieurs produits et un produit peut être acheté plusieurs fois

3. Règles de gestions de stocks :

- Un stock peut contenir un ou plusieurs produits, mais un produit est stocké dans un seul stock
- Dans un stock, il y a plusieurs emplacements, mais un emplacement est dans un seul stock

4. Règles de gestion des ventes :

- Une vente concerne un seul client et un client peut effectuer plusieurs achats (ventes)
- Une commande peut contenir plusieurs produits et un produit peut être vendu plusieurs fois
- Chaque commande est rattachée à un employé responsable et un employé peut effectuer plusieurs ventes

5. Règles d'organisations :

- Un employé peut gérer plusieurs emplacements et un emplacement peut être géré par plusieurs employés

IV. Dictionnaire de données :

Le dictionnaire de données représente l'ensemble des entités et attributs du modèle conceptuel de données (MCD), avec leurs fonctions. Nous distinguons les entités suivantes :

Produit : cette entité représente chaque article présent en stock ;

Espèce : elle décrit la nature biologique des végétaux ; Exples : "Lavandula angustifolia (Lavande), Rosa centifolia (Rose) ;

Catégorie : une catégorie est une famille de produits ; Exples : plantes d'intérieur, engrais, accessoires ;

Lot produit : c'est un groupe de produits par catégorie d'une même provenance ;

Stock : c'est la quantité disponible d'un produit dans un emplacement ;

Emplacement : c'est une zone de stockage dans le magasin où se trouve la localisation précise des produits ;

Fournisseur : c'est l'entreprise ou les particuliers qui fournissent les produits ;

Achat : elle représente l'approvisionnement des produits ;

Commande_client : cette commande représente la vente des produits ;

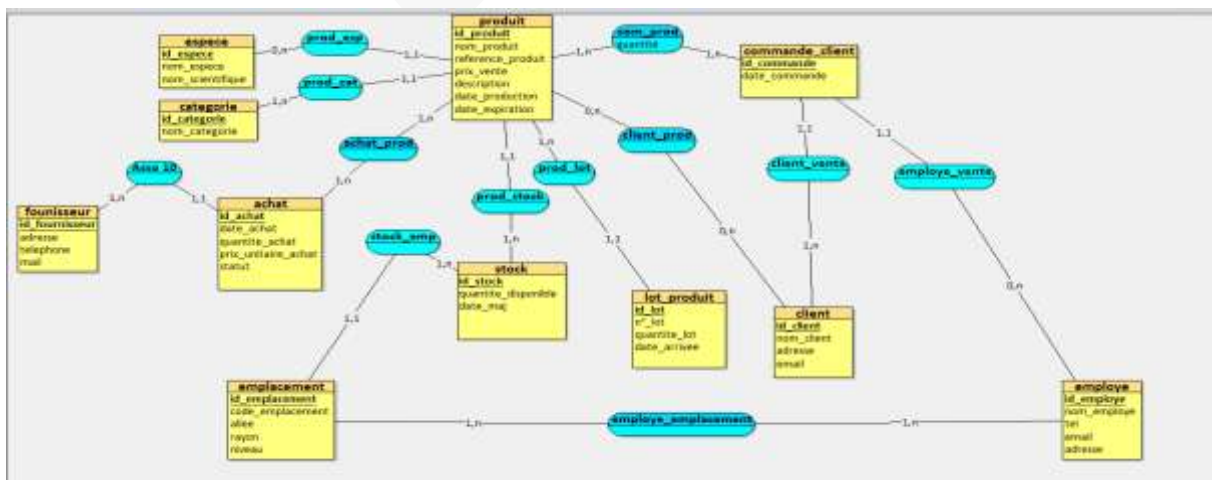
Client : représente l'entreprise ou les particuliers qui achètent des produits dans le magasin ;

Employé : comme son nom l'indique, c'est lui qui valide les ventes dans le magasin tout en s'occupant du mouvement du stock.

Ce sont ces entités qui représentent notre dictionnaire de données.

V. Modèle conceptuel de données(MCD) :

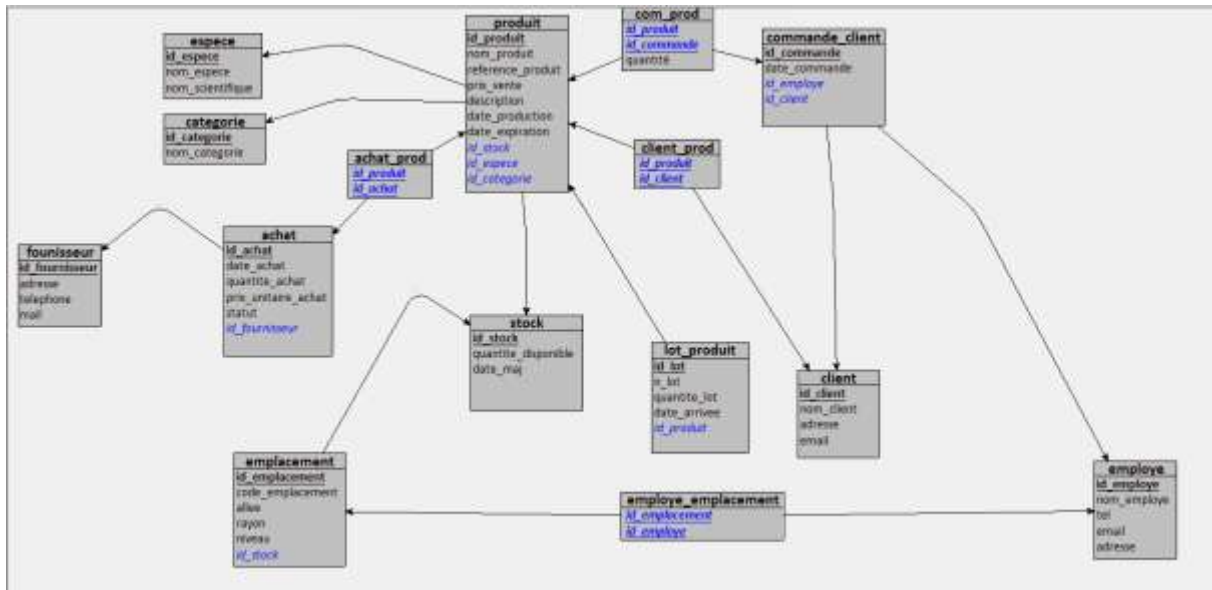
Dans le cadre du projet de gestion des végétaux, le MCD a pour objectif de modéliser les différentes entités telles que les clients, les produits, les fournisseurs, les catégories et les achats, ainsi que leurs interactions. Cette représentation facilite la compréhension globale du système d'information et garantit une cohérence dans l'organisation des données. On peut observer cette représentation ci-dessous :



Le Modèle Conceptuel de Données (MCD) proposé offre une représentation claire et structurée du système de gestion des végétaux. Il met en évidence les principales entités et leurs relations, tout en respectant les règles de gestion essentielles.

VI. Modèle logique de données(MLD) :

Le Modèle Logique de Données (MLD) constitue la traduction du Modèle Conceptuel de Données en une structure directement exploitable par un système de gestion de base de données relationnelle. Dans le cadre de ce projet, il permet de transformer les entités et associations du MCD en tables, en définissant les clés primaires et étrangères ainsi que les contraintes d'intégrité. On peut l'observer comme suit :



Le Modèle Logique de Données (MLD) présenté traduit fidèlement le MCD en une structure relationnelle organisée, composée de tables reliées par des clés primaires et étrangères. Il permet de garantir l'intégrité des données et d'assurer une gestion efficace des relations entre les différentes entités du système.

VII. Modèle physique de données(MPD) :

Le Modèle Physique de Données (MPD) représente l'étape finale de la conception d'une base de données. Il consiste à implémenter le Modèle Logique de Données dans un système de gestion de base de données, en tenant compte des contraintes techniques telles que les types de données, les index, les performances et les règles d'intégrité. Voici le script du projet :

```
CREATE TABLE categorie(
    id_categorie INT,
    nom_categorie VARCHAR(50),
    PRIMARY KEY(id_categorie)
);
```

```
CREATE TABLE espece(
```

```
id_espece INT,  
nom_espece VARCHAR(50),  
nom_scientifique VARCHAR(50),  
PRIMARY KEY(id_espece)  
);
```

```
CREATE TABLE stock(  
id_stock INT,  
quantite_disponible INT,  
date_maj DATE,  
PRIMARY KEY(id_stock)  
);
```

```
CREATE TABLE emplacement(  
id_emplacement INT,  
code_emplacement VARCHAR(50),  
allee INT,  
rayon INT,  
niveau INT,  
id_stock INT NOT NULL,  
PRIMARY KEY(id_emplacement),  
FOREIGN KEY(id_stock) REFERENCES stock(id_stock)  
);
```

```
CREATE TABLE fournisseur(  
id_fournisseur INT,  
adresse VARCHAR(50),  
telephone VARCHAR(50),  
mail VARCHAR(50),  
PRIMARY KEY(id_fournisseur)  
);
```

```
CREATE TABLE client(  
    id_client INT,  
    nom_client VARCHAR(50),  
    adresse VARCHAR(50),  
    email VARCHAR(50),  
    PRIMARY KEY(id_client)  
);
```

```
CREATE TABLE achat(  
    id_achat VARCHAR(50),  
    date_achat DATE,  
    quantite_achat INT,  
    prix_unitaire_achat DECIMAL(15,2),  
    statut VARCHAR(50),  
    id_fournisseur INT NOT NULL,  
    PRIMARY KEY(id_achat),  
    FOREIGN KEY(id_fournisseur) REFERENCES fournisseur(id_fournisseur)  
);
```

```
CREATE TABLE employe(  
    id_employe INT,  
    nom_employe VARCHAR(50),  
    tel VARCHAR(50),  
    email VARCHAR(50),  
    adresse VARCHAR(50),  
    PRIMARY KEY(id_employe)  
);
```

```
CREATE TABLE produit(  
    id_produit INT,  
    nom_produit VARCHAR(50),  
    reference_produit VARCHAR(50),
```

```
prix_vente DECIMAL(15,2),
description VARCHAR(50),
date_production DATE,
date_expiration DATE,
id_stock INT NOT NULL,
id_espece INT NOT NULL,
id_categorie INT NOT NULL,
PRIMARY KEY(id_produit),
FOREIGN KEY(id_stock) REFERENCES stock(id_stock),
FOREIGN KEY(id_espece) REFERENCES espece(id_espece),
FOREIGN KEY(id_categorie) REFERENCES categorie(id_categorie)
);
```

CREATE TABLE lot_produit(

```
id_lot INT,
n_lot INT,
quantite_lot INT,
date_arrivee DATE,
id_produit INT NOT NULL,
PRIMARY KEY(id_lot),
FOREIGN KEY(id_produit) REFERENCES produit(id_produit)
);
```

CREATE TABLE commande_client(

```
id_commande INT,
date_commande DATE,
id_employe INT NOT NULL,
id_client INT NOT NULL,
PRIMARY KEY(id_commande),
FOREIGN KEY(id_employe) REFERENCES employe(id_employe),
FOREIGN KEY(id_client) REFERENCES client(id_client)
);
```


CREATE TABLE client_prod(

id_produit INT,

id_client INT,

PRIMARY KEY(id_produit, id_client),

FOREIGN KEY(id_produit) REFERENCES produit(id_produit),

FOREIGN KEY(id_client) REFERENCES client(id_client)

);

CREATE TABLE achat_prod(

id_produit INT,

id_achat VARCHAR(50),

PRIMARY KEY(id_produit, id_achat),

FOREIGN KEY(id_produit) REFERENCES produit(id_produit),

FOREIGN KEY(id_achat) REFERENCES achat(id_achat)

);

CREATE TABLE employe_emplacement(

id_emplacement INT,

id_employe INT,

PRIMARY KEY(id_emplacement, id_employe),

FOREIGN KEY(id_emplacement) REFERENCES emplacement(id_emplacement),

FOREIGN KEY(id_employe) REFERENCES employe(id_employe)

);

CREATE TABLE com_prod(

id_produit INT,

id_commande INT,

quantité INT,

PRIMARY KEY(id_produit, id_commande),

FOREIGN KEY(id_produit) REFERENCES produit(id_produit),

FOREIGN KEY(id_commande) REFERENCES commande_client(id_commande));

VIII. Vérification et validation de la base de données :

Afin de vérifier la cohérence et le bon fonctionnement de la base de données, plusieurs scénarios de test ont été réalisés. Ces tests permettent de valider les contraintes d'intégrité, les relations entre les tables ainsi que la bonne insertion des données. Pour cela, nous avons préparé quelques use case notamment :

a. Ajout d'un nouveau client :

```
1 INSERT INTO client (id_client, nom_client, adresse, email)
2 VALUES (16, 'Diallo', 'Nantes', 'diallo@mail.com');
```

✓ 1 ligne insérée. (traitement en 0,0041 seconde(s).)

```
INSERT INTO client (id_client, nom_client, adresse, email) VALUES (16, 'Diallo', 'Nantes', 'diallo@mail.com');
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

b. Enregistrer un nouvel achat :

```
INSERT INTO achat (id_achat, date_achat, quantite_achat, prix_unitaire_achat, statut, id_fournisseur)
VALUES ('17', '2026-04-01', 20, 6.50, 'en cours', 2);
```

✓ 1 ligne insérée. (traitement en 0,0110 seconde(s).)

```
INSERT INTO achat (id_achat, date_achat, quantite_achat, prix_unitaire_achat, statut, id_fournisseur) VALUES ('17', '2026-04-01', 20, 6.50, 'en cours', 2);
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

c. Associer un produit à un achat :

```
1 INSERT INTO achat_prod (id_produit, id_achat)
2 VALUES (1, '16');
```

✓ 1 ligne insérée. (traitement en 0,0023 seconde(s).)

```
INSERT INTO achat_prod (id_produit, id_achat) VALUES (1, '16');
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

d. Modifier le statut d'un achat :

```
1 UPDATE achat
2 SET statut = 'livré'
3 WHERE id_achat = '16';
```

✓ 1 ligne affectée. (traitement en 0,0123 seconde(s).)

```
UPDATE achat SET statut = 'livré' WHERE id_achat = '16';
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

e. Supprimer un client :

```
1 DELETE FROM client
2 WHERE id_client = 16;
```

✓ 1 ligne supprimée. (traitement en 0,0039 seconde(s).)

```
DELETE FROM client WHERE id_client = 16;
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

f. Afficher tous les achats avec l'adresse du fournisseur :

```
1 SELECT a.id_achat, a.date_achat, a.quantite_achat, f.adresse
2 FROM achat a
3 JOIN fournisseur f ON a.id_fournisseur = f.id_fournisseur;
```

id_achat	date_achat	quantite_achat	adresse
1	2026-03-01	10	Fourn1
16	2026-04-01	20	Fourn1
17	2026-04-01	20	Fourn2
2	2026-03-02	12	Fourn2
3	2026-03-03	8	Fourn3
4	2026-03-04	15	Fourn4
5	2026-03-05	20	Fourn5
6	2026-03-06	9	Fourn6
7	2026-03-07	11	Fourn7
8	2026-03-08	14	Fourn8
9	2026-03-09	7	Fourn9
10	2026-03-10	13	Fourn10
11	2026-03-11	16	Fourn11
12	2026-03-12	18	Fourn12
13	2026-03-13	6	Fourn13
14	2026-03-14	10	Fourn14
15	2026-03-15	5	Fourn15

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table

g. Afficher les clients et leurs produits :

```
1 SELECT c.nom_client, p.nom_produit
2 FROM client c
3 JOIN client_prod cp ON c.id_client = cp.id_client
4 JOIN produit p ON cp.id_produit = p.id_produit;
```

nom_client	nom_produit
Dupont	Prod1
Martin	Prod2
Durand	Prod3
Petit	Prod4
Moreau	Prod5
Simon	Prod6
Laurent	Prod7
Michel	Prod8
Garcia	Prod9
David	Prod10
Bertrand	Prod11
Roux	Prod12
Vincent	Prod13
Fournier	Prod14
Girard	Prod15

☐ Tout afficher | Nombre de lignes : 25 ▼ | Filtrer les lignes:

h. Compter les achats par statut :

```

1 SELECT statut, COUNT(*) AS nombre
2 FROM achat
3 GROUP BY statut;

```

statut	nombre
livré	10
en cours	5
annulé	2

i. Calculer le chiffre d'affaire par client :

```

1 SELECT c.nom_client,
2        SUM(cp.quantité * p.prix_vente) AS chiffre_affai
3 FROM client c
4 JOIN commande_client cmd ON c.id_client = cmd.id_client
5 JOIN com_prod cp ON cmd.id_commande = cp.id_commande
6 JOIN produit p ON cp.id_produit = p.id_produit
7 GROUP BY c.nom_client;

```

nom_client	chiffre_affaires
Dupont	20.00
Martin	33.00
Durand	12.00
Petit	65.00
Moreau	28.00
Simon	60.00
Laurent	48.00
Michel	102.00
Garcia	38.00
David	19.00
Bertrand	140.00
Roux	63.00
Vincent	44.00
Fournier	115.00
Girard	96.00

j. Nombre de produits gérées par employé :

```
1 SELECT e.nom_employe, COUNT(DISTINCT cp.id_produit) AS nb_produits
2 FROM employe e
3 JOIN commande_client cmd ON e.id_employe = cmd.id_employe
4 JOIN com_prod cp ON cmd.id_commande = cp.id_commande
5 GROUP BY e.nom_employe;
```

nom_employe	nb_produits
Adam	1
Enzo	1
Hugo	1
Jean	1
Jules	1
Leo	1
Louis	1
Luc	1
Lucas	1
Marc	1
Nathan	1
Noah	1
Paul	1
Pierre	1
Tom	1

k. Produits les plus vendus avec leurs catégories :

```
1 SELECT p.nom_produit, c.nom_categorie, SUM(cp.quantité) AS total_vendu
2 FROM produit p
3 JOIN categorie c ON p.id_categorie = c.id_categorie
4 JOIN com_prod cp ON p.id_produit = cp.id_produit
5 GROUP BY p.nom_produit, c.nom_categorie
6 ORDER BY total_vendu DESC;
```

nom_produit	nom_categorie	total_vendu
Prod11	Orchidées	7
Prod8	Fougères	6
Prod4	Arbustes	5
Prod14	Carnivore	5
Prod6	Succulentes	4
Prod15	Médicinale	4
Prod2	Fleurs	3
Prod7	Herbes	3
Prod12	Palmier	3
Prod1	Plantes	2
Prod5	Graines	2
Prod9	Bonsaï	2
Prod13	Aquatique	2
Prod3	Arbres	1

NB : Chaque requête est accompagnée par son exécution dans MySQL.

IX. Conclusion :

Ce projet de conception de base de données a permis de mettre en œuvre une démarche structurée, allant du MCD au MPD, afin de modéliser efficacement la gestion des végétaux. L'utilisation de la méthode Merise a garanti une organisation cohérente des données et une bonne prise en compte des besoins métiers. La base obtenue est à la fois fiable, évolutive et adaptée aux opérations de gestion telles que les achats, les ventes et le suivi des stocks.

X. Bibliographie :

Afin de mener à la bonne réussite de ce projet, nous avons utilisé les notions vues pendant le cours ; et nous avons bénéficié de l'appui du professeur. Le déploiement de la base de données et le test des requêtes ont été fait dans MySQL. Le MCD, le MLD et le MPD ont été réalisés avec le logiciel looping.